Assignment 1
Classical Machine Learning methods

Part 1: Regressing Problems

Note:

1. Solve all the problems using MATLAB or any other computer language. However, try problem 1 by hand.
2. After calculating the best model given your data, review the provided data and you may remove any possible outliers (**a value with a possible error or high noise**), then recalculate the model for cleaned data. Compare the two models.
3. In all your answers to each question, write down the equations of your solutions (after calculating their parameters).
4. Hint: If the gain in R^2 is not high enough, it is better to take the model with a lower number of parameters.

1. The numbers of insured persons "y" with an insurance company for the years 1987 to 1996 are shown in the table.

Year	1987	1988	1989	1990	1991	1992	1993	1994	1995	1996
y	12400	10900	10000	1050	9500	8900	8000	7800	7600	7200

Make a scatterplot of the data, letting x represent the number of years since 1987.

a) Fit linear, quadratic, and cubic, by comparing the values of R^2. Determine the function that best fits the data. (Hint: take care of note 4 above)
b) You may review the data and remove what is outside the reasonable range (outlier), then recalculate the results and compare.
c) Graph the function of best fit with the scatterplot of the data.
d) With the best function found in part (b), predict the average number of people insured in 1997.

2. Develop a model for estimating heating oil used for a single-family home in January based on average temperature and amount of insulation in inches.

Oil	Temp F	Insulation
270	40	4
362	27	4
162	40	10
45	73	6
91	65	7
233	65	40
372	10	6
305	9	10
234	24	10
122	65	4
25	66	10
210	41	6
450	22	4
325	40	4
52	60	10

a) Fit linear, and quadratic functions to the data. By comparing the values R^2, determine the function that best fits the data.
b) Review the data and remove what is outside the reasonable range (outlier), then recalculate the results and compare.
c) Then use the regression models for the functions in b to predict the needed oil if the temperature is 15 Fahrenheit and the insulation is 5 attic insulations inches.

Part 2: some classical ML modeling

Use the ReducedMNIST which is a reduced version of the MNIST data set.

- **ReducedMNIST training**: 1000 examples for each digit.
- **ReducedMNIST test**: 200 examples for each digit.

1. Use the ReducedMNIST data to generate these features for each of the images of the training and testing sets:
   a. DCT features (225 dimensions)
   b. PCA (use several dimensions so that the total variance is at least 95% of the total variance when using all the 784 dimensions).
   c. HOG (Histogram of Oriented Gradients): Extract HOG features from each image, which encode local edge and gradient structure. This representation is more invariant to minor pixel-level variations and tends to produce more class-coherent clusters than raw pixels. The MATLAB function is: extractHOGFeatures.
2. Then train these classifiers using the training set of the **ReducedMNIST** data for each of the above features:
   a. K-means for each class. Try 1, 4, 16, and 32 clusters for each class.
   b. SVM. You may try the linear and the nonlinear kernels (like RBF). In this case **state clearly** what kernel you have used.

Then use the resulting models to classify the test set. Compare the different features and the different classifiers:
You must add a final table to summarize all your results (accuracy and processing time) comparatively, as in the table that is shown below.

1. **Only for the best result of each classifier show it in a confusion matrix among the 10 digits.**
2. **Add your conclusions.**

		Features					
		DCT		PCA		HOG	
		Accuracy	Processing Time	Accuracy	Processing Time	Accuracy	Processing Time
Classifier							
K-means Clustering	1						
	4						
	16						
	32						
SVM	Linear						
	nonlinear*						

* Mention the kernel name and its specs

Part 3: ways to fasten the Labelling Process

Problem Statement:

Manual data labelling is both costly and time-consuming. This assignment explores semi-automatic labelling methods that minimize human involvement, thereby reducing the overall labelling effort and cost.

You are given the Indian_Digits_Train dataset — a collection of 10,000 unlabelled grayscale images spanning 10 classes representing the Indian digits 0 through 9 (1,000 examples each). The dataset is organized as a single folder containing 10,000 .bmp files, named sequentially from 1.bmp to 10000.bmp. Each image is 28×28 pixels. Your task is to design a labelling pipeline: a structured series of steps that automatically assigns a digit label (0–9) to each image with minimal human **intervention.**

Labelling Accuracy Requirement

Your pipeline must achieve a labelling accuracy of at least 99%. This threshold is not arbitrary — in downstream machine learning workflows, the quality of training labels directly determines model performance. Even a seemingly small error rate of 1% introduces 100 mislabeled examples into this dataset, which can measurably degrade a classifier's generalization ability, particularly near decision boundaries between visually similar classes.

Measuring Labelling Accuracy

Since you cannot rely on visual inspection alone to verify 10,000 labels, two complementary evaluation methods are provided:

Oracle Ground Truth: A MATLAB function, provided as a ".p" file named check_accuracy, accepts a 10,000×1 vector of predicted labels — one integer per image, ordered by filename (i.e., the first element corresponds to 1.bmp, the second to 2.bmp, and so on) — and returns the achieved labelling accuracy. This gives you an exact, objective measure of your pipeline's performance. Note that this method is only possible here because the true labels are known to the instructor; in real-world scenarios, such an oracle is rarely available.

Practical Ground Truth: To simulate a realistic evaluation setting, a human annotator manually labels 5% of the dataset (500 images). This labelled subset serves as a held-out validation set for estimating your pipeline's accuracy without access to the full ground truth — the kind of evaluation you would rely on in practice.

You are encouraged to use both methods during development: the oracle to rigorously tune and debug your pipeline, and the practical method to appreciate the limitations and uncertainties of real-world label verification.

Comparing Pipelines: Manual Labelling Time

Different pipelines will require different amounts of human involvement to reach the 99% accuracy target. The key metric for comparing them is the **total manual labelling time** — that is, the number of images a human must inspect or verify multiplied by the time required per image. For this assignment, assume that **classifying a single image takes 10 seconds**.

As a reference point, if a human were to label the entire dataset manually with no automation, this would require:

$$10{,}000 \text{ images} \times 10 \text{ seconds} = 100{,}000 \text{ seconds} \approx 27.8 \text{ hours}$$

A good pipeline should dramatically reduce this figure while still meeting the 99% accuracy requirement. When reporting your results, you must state the **number of images manually handled** and the corresponding **total manual time** your pipeline requires. This allows a fair, quantitative comparison across different pipeline designs.

Labelling Pipelines

Below are some example pipelines that can help reduce manual labelling effort. You are free to modify these pipelines or propose your own; if you deviate from the examples, clearly document your changes and justify your design choices.

.

Pipeline 1: K-Means Bootstrapping with Active SVM Refinement

This pipeline combines unsupervised clustering, human-in-the-loop label bootstrapping, and iterative SVM refinement with weighted retraining. The key idea is to spend human effort wisely: first by labelling entire clusters at once rather than individual images, and then by focusing human attention only on the most ambiguous examples near the classifier's decision boundaries.

Step 1 — Feature Extraction and K-Means Clustering

Before clustering, you may optionally apply one of the following feature extraction methods to improve cluster coherence. Raw pixel vectors in 784 dimensions can be noisy and sensitive to stroke variation, so dimensionality reduction or feature engineering can significantly improve clustering quality:

- No preprocessing: Flatten each 28×28 image into a 784-dimensional raw pixel vector and cluster directly. Then you need to apply one of the given feature extraction methods: DCT, PCA, or HOG feature.

MATLAB functions: dct, pca, extractHOGFeatures. You may use one of them.

Regardless of the chosen representation, apply K-means clustering to all 10,000 feature vectors using K = 40 to 80 clusters — that is, roughly 4 to 8 clusters per digit class on average. The rationale for using more clusters than the number of classes is to keep each cluster internally coherent: a cluster should ideally contain images of a single digit, making human label assignment reliable in the next step.

MATLAB function: kmeans

Step 2 — Human Labelling of Clusters (Bootstrapping Phase)

For each cluster, present a human annotator with 8 randomly sampled images from that cluster simultaneously. The annotator assigns a single majority label to the entire cluster in one decision. This counts as 20 seconds of manual time per cluster, regardless of cluster size, since all 8 images are judged together in a single view.

Important: If the 8 sampled images appear to belong to multiple digit classes, do not force a single label onto the cluster. Instead, flag the cluster for splitting or discard it from the initial seed set, as a mixed cluster will corrupt the bootstrapped labels.

Propagate the assigned label to all images in the cluster. This produces an initial set of noisy but reasonably consistent labels for the full dataset at a fraction of the cost of individual labelling.

Estimated manual time for this step:

- At 40 clusters: 40 × 20 s = 800 s ≈ 13.3 minutes
- At 80 clusters: 80 × 20 s = 1,600 s ≈ 26.7 minutes

Compare this against the 27.8-hour baseline of fully manual labelling.

Step 3 — Initial SVM Training

Train a multi-class SVM classifier using the K-means-derived labels from Step 2. Use a Radial Basis Function (RBF) kernel within a one-vs-one multi-class scheme. This initial classifier will achieve some labelling accuracy, but not necessarily the 99% target.

MATLAB functions: fitcecoc, templateSVM with 'KernelFunction', 'rbf'

Step 4 — Active Refinement: Labelling Boundary Images

Identify the 20–40 images that the SVM is least confident about by computing the decision function scores for all images. Use the predict function with score output:

matlab

```
[predictedLabels, scores] = predict(SVMModel, features);
```

Each row of scores contains the decision function values across all one-vs-one binary classifiers for that image. A useful confidence measure is the margin — the difference between the highest and second-highest score. Images with the smallest margin are the most ambiguous and sit closest to the decision boundaries. Select the 20–40 images with the smallest margin values and present them to a human annotator for manual labelling, at the standard rate of 10 seconds per image.

Once labelled, update the training set as follows:

- Replace the K-means-derived label for each boundary image with the human-assigned label.
- Assign these human-labelled images a sample weight of 100, compared to a weight of 1 for all K-means-labelled images. This forces the SVM to strongly respect the trusted human labels during retraining. The weight value of 100 is a tuneable hyperparameter; you are encouraged to experiment with different values and observe the effect on convergence speed.

Estimated manual time per iteration: 20–40 images × 10 s = 200–400 seconds

Step 5 — Retrain and Evaluate

Retrain the SVM classifier on the updated label set, passing the sample weights via the 'Weights' option in fitcecoc. Evaluate the new labelling accuracy using the check_accuracy oracle and/or the practical held-out set. Record the improvement in accuracy relative to the previous iteration.

Step 6 — Iterate Until Convergence

Repeat Steps 4–5 until one of the following stopping conditions is met:

- The labelling accuracy reaches or exceeds 99%, or
- The improvement in accuracy between consecutive iterations falls below a defined threshold (e.g., < 0.1%), indicating convergence.

Reporting Requirements

At the end of the pipeline, report (**put the required info in a block diagram and a compact table for ease of readability**):

- Draw a block diagram to show the exact steps and values you have chosen in this pipeline.
- The feature representation used (raw pixels, DCT, PCA, or HOG) and, if applicable, the number of PCA components retained
- Final labelling accuracy (oracle and/or practical)
- Number of iterations performed
- Total number of images manually handled (clusters × 8 images viewed + boundary images labelled across all iterations)
- Total estimated manual time

Pipeline 2: Manual Seed Labelling with Augmentation and Self-Training Refinement

This pipeline takes a different approach from Pipeline 1. Rather than clustering the full dataset first, it begins by manually labelling a small random seed set, expands it through data augmentation, and then grows the labelled pool iteratively by combining active learning on uncertain boundary examples with self-training on high-confidence predictions. The key idea is to start with a trusted, human-labelled core and let the classifier progressively earn the right to label more data on its own.

Step 1 — Random Seed Sampling and Manual Labelling

Randomly sample 300 images from the unlabelled dataset — approximately 30 images per class, assuming a roughly uniform class distribution. A human annotator manually assigns the correct digit label (0–9) to each of these images at the standard rate of 10 seconds per image.

Estimated manual time for this step: 300 images × 10 s = 3,000 s ≈ 50 minutes

Compare this against the 27.8-hour baseline of fully manual labelling.

These 300 images form the trusted seed set and will be assigned a sample weight of 100 throughout all subsequent training steps to ensure the classifier always strongly respects human-verified labels.

Step 2 — Data Augmentation

To compensate for the small size of the seed set and improve the robustness of the initial classifier, augment each of the 300 manually labelled images by applying the following transformations:

- Rotation: ±5° (two additional copies per image)
- Additive noise: Apply low-level Gaussian noise to simulate handwriting variability
- Spatial shifts: Shift the digit within the 28×28 frame by a few pixels in each direction (up, down, left, right)

Each transformation produces one additional image, so each original image can yield several augmented copies. The augmented images inherit the label of their source image and are assigned a sample weight of 1 — the same as pseudo-labelled images added in later steps, since augmented images, while derived from trusted labels, are synthetically generated and should not be given disproportionate influence over the classifier.

MATLAB functions: imrotate, imnoise, imtranslate

Step 3 — Initial SVM Training (SVM-1)

Train a multi-class SVM classifier (SVM-1) on the combined seed set (original 300 images with weight = 100, plus augmented copies with weight = 1), using a Radial Basis Function (RBF) kernel within a one-vs-one multi-class scheme. Use SVM-1 to generate predicted labels and decision function scores for all 10,000 images in the dataset:

matlab

```
[predictedLabels, scores] = predict(SVM1Model, features);
```

This initial classifier establishes a first estimate of the full dataset's labels, but accuracy at this stage will likely fall short of the 99% target.

Step 4 — Active Refinement: Labelling Boundary Images

Identify the 20 images that SVM-1 is least confident about using the margin — the difference between the highest and second-highest score in each row of scores. Images with the smallest margin sit closest to the decision boundaries and are the most likely to be mislabelled by the classifier.

Present these 20 images to a human annotator for manual labelling at 10 seconds per image. Once labelled:

- Add these images to the training set with a sample weight of 100, matching the original seed set.

Estimated manual time per iteration: 20 images × 10 s = 200 seconds

Step 5 — Self-Training: Incorporating High-Confidence Predictions

In addition to the human-labelled boundary images, expand the training set by selecting the 50 images per class with the highest margin scores, subject to a minimum margin threshold. Specifically, only include an image if its margin exceeds the 75th percentile of all margin scores in the current iteration — this ensures that only genuinely confident predictions are incorporated, and guards against early high-confidence but incorrect predictions being reinforced over subsequent iterations.

Images that pass this threshold receive pseudo-labels from the current classifier's predictions and are added to the training set with a sample weight of 1.

This step adds up to 500 new pseudo-labelled images per iteration (50 × 10 classes, subject to the threshold), allowing the classifier to leverage its own most reliable predictions to grow the training set without any additional human effort.

Important: Only add images not already present in the training set. Maintain a running index of all images incorporated so far to avoid duplicate entries across iterations.

Step 6 — Retrain and Evaluate (SVM-2, SVM-3, ...)

Retrain the SVM classifier on the updated training set — which now includes the original seed set, augmented images, human-labelled boundary images, and high-confidence pseudo-labelled images — passing all sample weights via the 'Weights' option in fitcecoc. Evaluate the new labelling accuracy using the check_accuracy oracle and/or the practical held-out set. Record the improvement in accuracy relative to the previous iteration.

MATLAB functions: fitcecoc, templateSVM with 'KernelFunction', 'rbf'

Step 7 — Iterate Until Convergence

Repeat Steps 4–6 until one of the following stopping conditions is met:

- The labelling accuracy reaches or exceeds 99%, or
- The improvement in accuracy between consecutive iterations falls below a defined threshold (e.g., < 0.1%), indicating convergence.

Reporting Requirements

At the end of the pipeline, report (**put the required info in a block diagram and a compact table for ease of readability**):

- Draw a block diagram to show the exact steps and values you have chosen in this pipeline.
- Final labelling accuracy (oracle and/or practical)
- Number of iterations performed
- Total number of images manually labelled (300 seed + 20 boundary images × number of iterations)
- Total number of pseudo-labelled images incorporated, and the number rejected by the margin threshold each iteration
- Total estimated manual time
- A comparison of labelling accuracy and manual time against Pipeline 1

Pipeline 3: LLM-Based Automatic Labelling with Agreement Validation

This pipeline takes a fundamentally different approach from Pipelines 1 and 2. Rather than training a classifier from scratch, it leverages pre-trained Large Language Models (LLMs) as zero-shot labellers — feeding each image directly to multiple LLMs and using inter-model agreement as a proxy for label confidence. Human effort is reserved exclusively for the practical ground truth set that is already required by the assignment, disagreement resolution, and corrective refinement. The key idea is that when two independently selected, high-performing LLMs agree on a label, that agreement is a strong signal of correctness.

Step 1 — LLM Benchmarking on the Practical Ground Truth Set

Rather than labelling a separate evaluation set, use the 500-image practical ground truth set that you have already manually labelled as part of the assignment requirements. This set serves double duty: as the standard accuracy measurement tool for the assignment, and as the benchmarking reference for selecting the best LLMs in this step. No additional manual labelling is required here.
Select five candidate LLMs capable of image classification (i.e., with vision input support). Have each LLM independently label all 500 images using a consistent prompt that describes the task: classifying a 28×28 grayscale image of a handwritten Indian digit into one of 10 classes (0–9). Measure each LLM's labelling accuracy against the 500-image ground truth.
Prompt design matters significantly here. A well-structured prompt should include:

- A clear description of the digit classes (0–9)
- A statement that the images are handwritten Indian digits
- An instruction to respond with only the predicted digit and nothing else, to simplify parsing
- Optionally, one or two few-shot examples to anchor the LLM's understanding of the visual style

Note on image resolution: Some LLMs may perform poorly on very small, low-resolution grayscale images, particularly if they were predominantly trained on natural photographs. Consider upscaling each image (e.g., to 224×224 pixels) before passing it to the LLM, which may significantly improve accuracy at negligible additional cost.
Estimated manual time for this step: 0 additional seconds — the 500-image set is already labelled as part of the assignment.

Step 2 — Select the Two Top-Performing LLMs

Rank the five LLMs by their accuracy on the 500-image ground truth set and select the two with the highest accuracy. These two models will be used to label the full dataset. If two or more models are tied, prefer the one with higher confidence scores or lower response latency, depending on operational constraints.
Important: If the top two LLMs both achieve low accuracy on the benchmarking set (e.g., below 90%), this pipeline may not be viable without prompt refinement first. In that case, proceed directly to the corrective actions described in Step 5 before attempting full-scale labelling.

Step 3 — Full Dataset Labelling by Both LLMs

Have each of the two selected LLMs independently label all 10,000 images in the dataset. The two models operate entirely separately at this stage — neither is aware of the other's predictions. This independence is essential for the agreement-based validation in the next step to be meaningful. For the pipeline to be statistically valid, the two LLMs should ideally be drawn from different model families (e.g., one from the GPT family and one from the Gemini or Claude family) to minimize the risk of correlated errors stemming from shared training data or architecture.
No manual time is incurred in this step. However, depending on the LLMs used, this step may incur API costs. Estimate and report the computational cost of labelling 10,000 images with two LLMs as part of your reporting requirements.

Step 4 — Agreement-Based Validation and Disagreement Resolution

Partition the 10,000 images into two groups based on whether the two LLMs produced the same label:
4a. Agreed Labels: For images where both LLMs assign the same label, evaluate their accuracy using the 500-image practical ground truth set wherever those images overlap — no additional manual labelling is needed. If the verified accuracy meets or exceeds 99%, accept all agreed labels as final.
This approach gives a statistically stronger accuracy estimate than a freshly sampled spot-check, at zero additional manual cost.
4b. Disagreed Labels: For images where the two LLMs assign different labels, isolate them and present each to a human annotator for manual labelling at 10 seconds per image. These human-assigned labels are accepted as final for this subset.
Estimated manual time for this sub-step: N_disagree × 10 s, where N_disagree is the number of disagreements determined empirically.
The total manual time for this pipeline is therefore highly sensitive to the disagreement rate. If the two LLMs disagree on, say, 5% of the dataset (500 images), the manual cost for disagreement resolution is 500 × 10 s = 5,000 s ≈ 83 minutes. A higher-quality LLM pair will reduce this figure significantly.

Step 5 — Corrective Actions if Target Accuracy Is Not Achieved

If the accuracy measured in Step 4a falls below 99%, do not accept the agreed labels. Instead, consider one or more of the following corrective actions before repeating Steps 3–4:

- Prompt refinement: Revise the labelling prompt to include clearer class descriptions, more explicit formatting instructions, or additional context about the visual characteristics of Indian digits.
- Few-shot examples: Provide one or two verified example images with their correct labels directly in the prompt to anchor the LLM's predictions to the correct visual style.
- Tie-breaker LLM: Introduce the third-ranked LLM from Step 2 as a tie-breaker for disagreements — replacing manual annotation for that subset and reserving human effort only for three-way disagreements.
- **Active learning fallback**: If accuracy remains below target after prompt refinement, fall back to the boundary-labelling strategy from Pipelines 1 or 2 — use the LLM-generated labels as a noisy seed set and refine iteratively with a weighted SVM.

Reporting Requirements

At the end of the pipeline, report (**put the required info in a block diagram and a compact table for ease of readability**):

- Draw a block diagram to show the exact steps and values you have chosen in this pipeline.
- Accuracy of each of the five LLMs on the 500-image benchmarking set (Step 1)
- Final labelling accuracy of the two selected LLMs on the agreed subset (Step 4a)
- Number of disagreements (N_disagree) and the manual time spent resolving them
- Total number of images manually handled and total estimated manual time across all steps
- Estimated computational and API cost of running two LLMs over 10,000 images
- A comparison of labelling accuracy and manual time against Pipelines 1 and 2

Summarize your results for pipeline 3 in this table

Pipeline 1	Manual time	Accuracy
iteration-1 results		
iteration-2 results		
iteration-3 results		
Pipeline 2		
iteration-1 results		
iteration-2 results		
iteration-3 results		
Pipeline 3		
LLM-1 alone		
LLM-2 alone		
LLM-1 & LLM-2		